

Internship and Job Tracking Dashboard

Tools & Techniques for Data Science - Assignment 4 Report

Submitted By

Name	Roll Number	GitHub Username
Maryam	l1f23bsds0039	maryamdatascientist
Fatima	l1f23bsds0040	fatima-5671
Aniqa	l1f23bsds0010	Aniqa-r

GitHub Repository

<https://github.com/maryamdatascientist/streamlit-postgres-assignment>

<https://github.com/Aniqa-r/streamlit-postgres-assignment>

<https://github.com/fatima-5671/streamlit-postgres-assignment.git>

1. Introduction

This project is an Internship and Job Tracking Dashboard developed for a university department. The purpose of the system is to help faculty members manage internship and job opportunities for students through a simple web-based dashboard.

The application allows users to add new opportunities, view and search records, update existing records, delete invalid records, upload data using CSV files, export records, detect duplicate opportunities, view deadline alerts, and analyze opportunity trends through visual dashboards.

The project was developed using Streamlit, PostgreSQL, pgAdmin, Docker Compose, and GitHub. PostgreSQL is used as the permanent database, Streamlit is used for the user

interface, pgAdmin is used for database verification, Docker Compose is used for running all services together, and GitHub is used for version control and contribution tracking.

2. Problem Statement

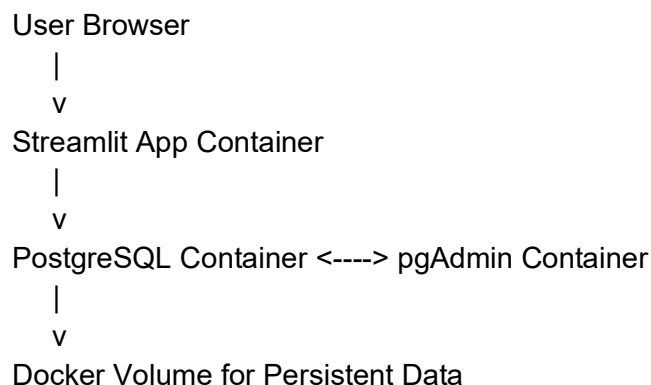
The university department receives internship and job records from multiple companies. Managing this data manually can create several problems, such as duplicate entries, missing deadlines, difficulty searching records, lack of analytics, and difficulty verifying stored data.

To solve this problem, a complete data application was developed. The system stores records in PostgreSQL and provides an interactive Streamlit interface for faculty members. The application supports CRUD operations, CSV upload/export, duplicate detection, deadline alerts, analytics, and role-based access control.

3. System Architecture

The project uses a multi-container Docker architecture. The user accesses the Streamlit application from the browser. The Streamlit application communicates with the PostgreSQL database container. pgAdmin is also connected to the PostgreSQL container for database inspection and SQL verification. PostgreSQL data is stored in a Docker volume to preserve data.

Architecture Diagram



GitHub Repository stores source code, commits, screenshots, README, and report.

Main Components

Component	Description
Streamlit App	Provides frontend pages and dashboard
PostgreSQL	Stores internship and job opportunity records
pgAdmin	Used to inspect database tables and run SQL queries
Docker Compose	Runs PostgreSQL, pgAdmin, and Streamlit together
GitHub	Stores source code and contribution history

4. Database Design

The PostgreSQL database is named:

student_opportunities_db

The main table is named:

opportunities

The table stores all internship and job opportunity records.

Main Table Columns

Column	Description
opportunity_id	Primary key
company_name	Name of the company

job_title	Internship or job title
category	Category such as Data Science, AI, Web Development, Cyber Security, Software Engineering
city	City where the opportunity is available
country	Country
work_mode	Remote, Onsite, or Hybrid
required_skills	Required skills for the opportunity
salary_min	Minimum salary
salary_max	Maximum salary
currency	Currency such as PKR, USD, or EUR
experience_level	Internship, Entry Level, Mid Level, or Senior Level
application_deadline	Last date to apply
status	Open, Closed, Expired, or Shortlisted
source_link	Original opportunity source URL
created_at	Timestamp of record creation

Database Scripts

The database structure is created using:

database/init.sql

Sample data is inserted using:

database/seed_data.sql

The project includes at least 40 records with multiple companies, cities, categories, work modes, salary ranges, currencies, and statuses.

5. Docker Compose Explanation

Docker Compose is used to run the complete project environment using three services.

Services

Service Name	Container Name	Port	Purpose
postgres_db	opportunity_postgres	5432:5432	Runs PostgreSQL database
pgadmin	opportunity_pgadmin	5050:80	Runs pgAdmin database GUI
streamlit_app	opportunity_streamlit	8501:8501	Runs Streamlit web application

Docker Concepts Used

services

The `services` section defines the containers required for the project.

image

The PostgreSQL and pgAdmin services use official Docker images.

build

The Streamlit app is built using the Dockerfile.

container_name

Each container has a readable name, such as `opportunity_postgres`.

ports

Ports are mapped from the container to the local machine.

environment

Environment variables are used for database credentials and connection settings.

volumes

A named volume is used to preserve PostgreSQL data.

depends_on

The Streamlit and pgAdmin services depend on the PostgreSQL service.

restart policy

The restart policy keeps services running unless stopped manually.

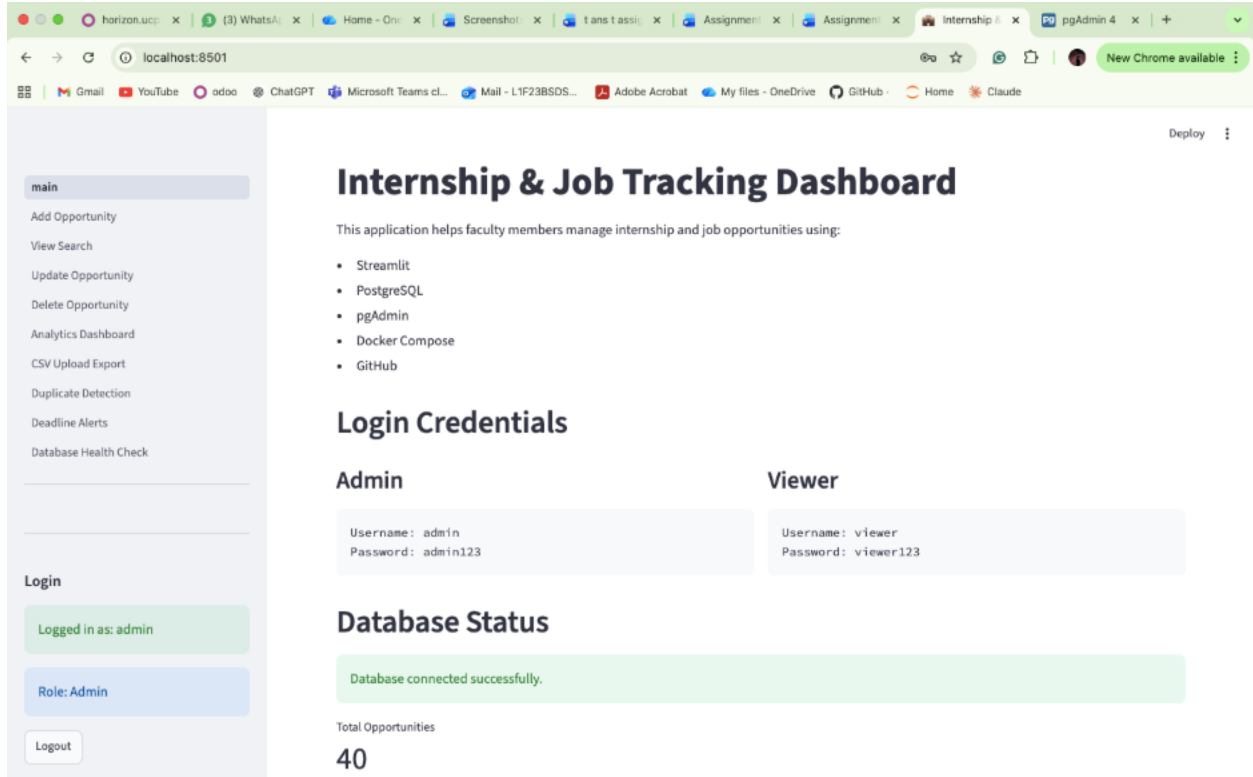
Docker network communication

Inside Docker Compose, containers communicate using service names. For example, pgAdmin connects to PostgreSQL using the host name `postgres_db`.

6. Streamlit Application Screenshots

Home Page

The Home page shows the project title, login credentials, database connection status, and total number of records.



Add Opportunity Form

The Add Opportunity page allows Admin users to insert new internship or job records with validation.

horizon.uce x (3) WhatsApp x Home - Oni x Screenshot x t ans t assi x Assignment x Assignment x + Add New O x pgAdmin 4 x +

localhost:8501/Add_Oppportunity

Deploy

main

Add Opportunity

View Search

Update Opportunity

Delete Opportunity

Analytics Dashboard

CSV Upload Export

Duplicate Detection

Deadline Alerts

Database Health Check

Login

Logged in as: admin

Role: Admin

Logout

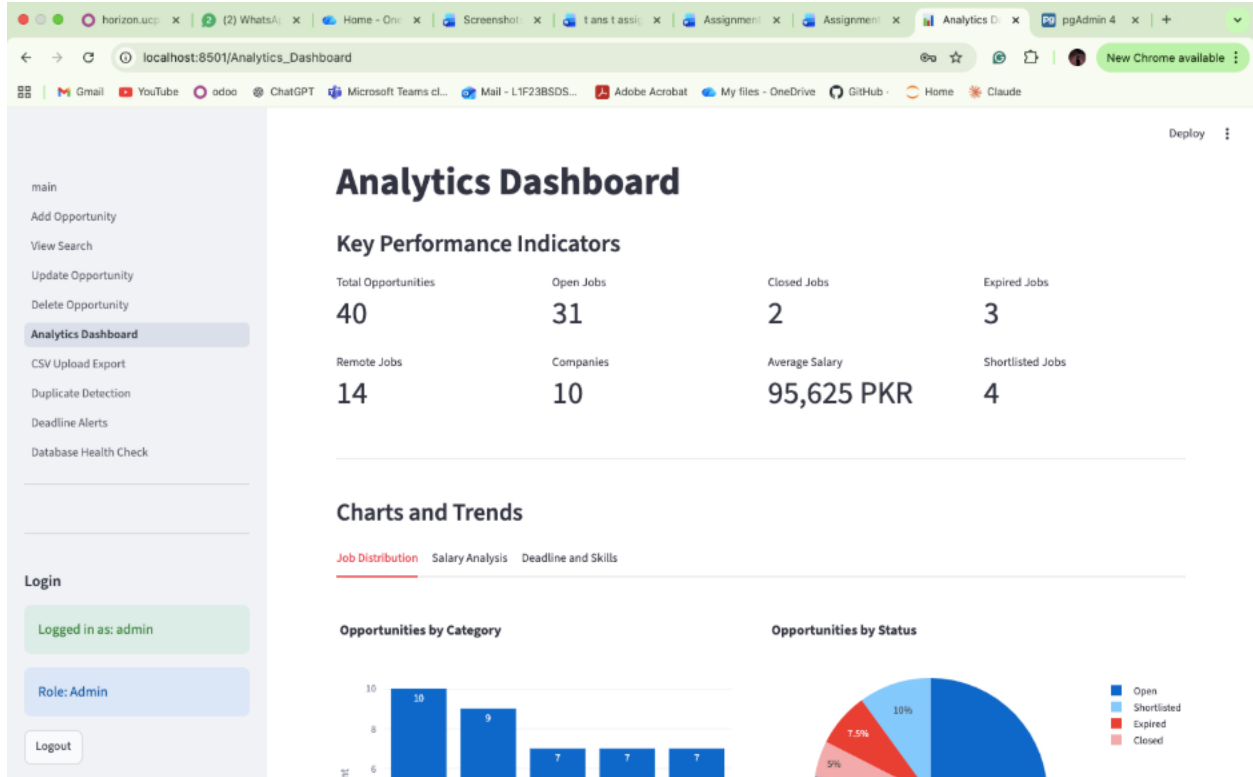
Add New Opportunity

Use this form to add a new internship or job opportunity into PostgreSQL.

Company Name *	Minimum Salary
	0.00 - +
Job Title *	Maximum Salary
	0.00 - +
Category *	Currency
Data Science	PKR
City	Application Deadline
	2026/06/14
Country	Status
Pakistan	Open
Work Mode *	Source Link
Remote	

Analytics Dashboard

The Analytics Dashboard shows KPIs and visual charts for opportunity analysis.



7. CRUD Operations Explanation

The application supports complete CRUD operations.

Create

Admin users can add a new opportunity through the Add New Opportunity page. The form includes validation for required fields, salary range, work mode, status, and duplicate checking.

Read

Users can view all records on the View and Search page. The records are loaded from PostgreSQL and displayed using Streamlit dataframes.

Update

Admin users can select an existing opportunity and update fields such as status, deadline, salary, skills, work mode, and source link.

Delete

Admin users can preview a selected record and delete it only after confirmation. This prevents accidental deletion.

8. CSV Upload and Export Explanation

The CSV Upload / Export page supports bulk data operations.

CSV Upload

Admin users can upload a CSV file containing opportunity records. The application validates required columns and checks field values such as category, work mode, status, currency, salary range, and deadline date.

If duplicate records are found, they are skipped.

CSV Export

Users can filter records and download the filtered data as a CSV file. This allows faculty members to share or analyze selected opportunity records outside the application.

9. Login and Role Management Explanation

The application includes a basic login system using Streamlit session state.

Roles

Role	Access
Admin	Add, update, delete, CSV upload/export, view dashboard, duplicate detection, deadline alerts

Viewer View/search records, analytics dashboard, duplicate detection, deadline alerts, database health check

Credentials Used

Admin

Username: admin
Password: admin123

Viewer

Username: viewer
Password: viewer123

The Viewer role cannot access Admin-only pages such as Add, Update, Delete, and CSV Upload/Export.

10. Dashboard Explanation

The Analytics Dashboard provides KPIs and charts to analyze internship and job opportunity trends.

KPIs

The dashboard includes the following KPIs:

1. Total opportunities
2. Open jobs
3. Closed jobs
4. Expired jobs
5. Remote jobs
6. Number of companies
7. Average salary
8. Shortlisted jobs

Charts

The dashboard includes multiple charts, such as:

1. Opportunities by category
2. Opportunities by status
3. Work mode distribution
4. Opportunities by city
5. Average salary by category
6. Salary distribution by experience level
7. Application deadlines by month
8. Top required skills

These charts help faculty members understand hiring trends, popular job categories, work modes, salary ranges, and important deadlines.

11. pgAdmin Verification

pgAdmin was used to verify the PostgreSQL database, inspect the `opportunities` table, and run SQL queries.

pgAdmin Connection Details

Host name/address: postgres_db

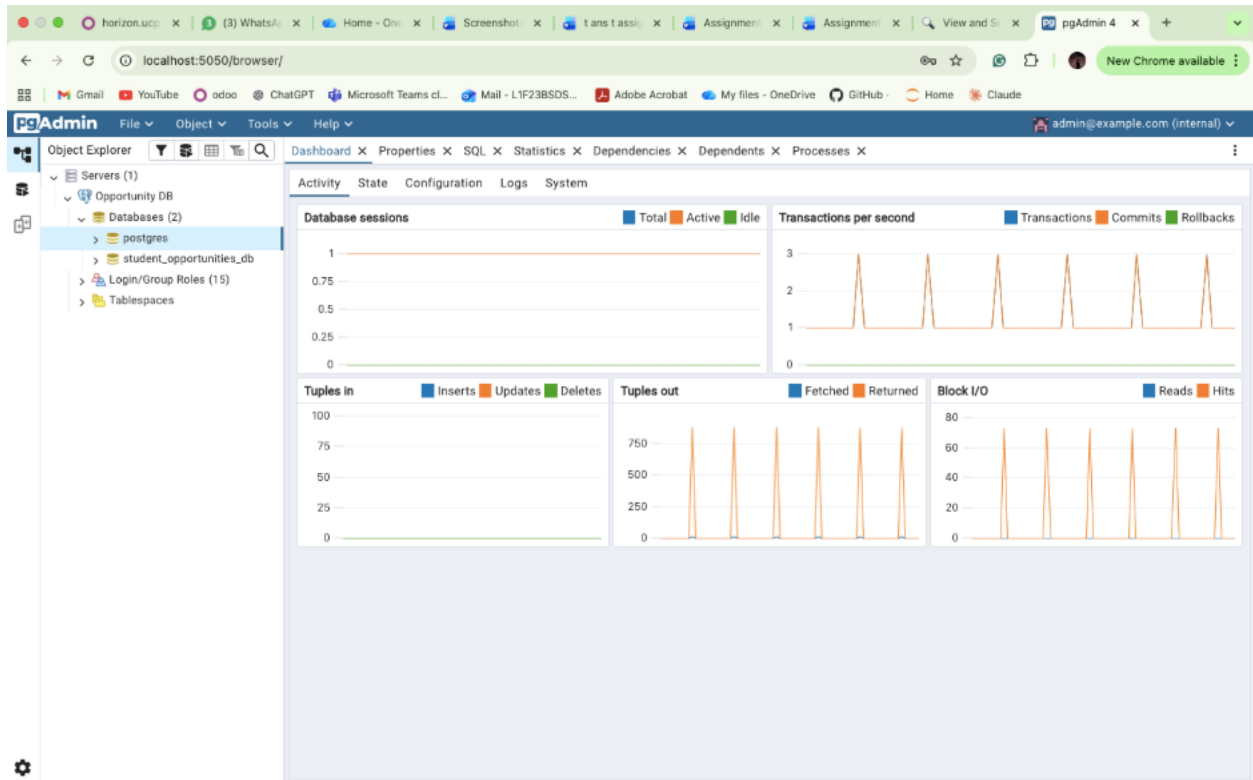
Port: 5432

Maintenance database: student_opportunities_db

Username: app_user

Password: app_password

pgAdmin Connection Screenshot



PostgreSQL Table Screenshot

The screenshot shows the pgAdmin 4 interface with the 'SQL' tab selected. The 'Query' window displays the following SQL query:

```
SELECT * FROM public.opportunities
ORDER BY opportunity_id ASC
```

The 'Data Output' window shows the results of the query, displaying 40 rows. The table has the following columns:

opportunity_id [PK] integer	company_name character varying (100)	job_title character varying (150)	category character varying (50)	city character varying (80)	country character varying (80)	work_mode character varying (30)	req text
1	Google	Data Analyst Intern	Data Science	Lahore	Pakistan	Remote	Py
2	Microsoft	Junior Data Scientist	Data Science	Karachi	Pakistan	Hybrid	Py
3	IBM	AI Research Intern	AI	Islamabad	Pakistan	Onsite	Py
4	Systems Ltd	Web Developer Intern	Web Development	Lahore	Pakistan	Onsite	HT
5	Arbisoft	Software Engineer Trainee	Software Engineering	Lahore	Pakistan	Hybrid	Py
6	NETSOL	Cyber Security Analyst Intern	Cyber Security	Lahore	Pakistan	Onsite	No
7	10Pearls	Machine Learning Intern	AI	Karachi	Pakistan	Remote	Py
8	Afiniti	Data Engineer Associate	Data Science	Islamabad	Pakistan	Hybrid	SQ
9	Contour Software	Frontend Developer	Web Development	Karachi	Pakistan	Hybrid	Re
10	Tkxel	Backend Developer Intern	Software Engineering	Lahore	Pakistan	Onsite	No
11	Google	Cloud Security Intern	Cyber Security	Islamabad	Pakistan	Remote	Cit
12	Microsoft	AI Product Intern	AI	Lahore	Pakistan	Remote	Py
13	IBM	Database Administrator Trainee	Software Engineering	Karachi	Pakistan	Onsite	SQ
14	Systems Ltd	Business Intelligence Analyst	Data Science	Islamabad	Pakistan	Hybrid	Po
15	Arbisoft	React Developer Intern	Web Development	Faisalabad	Pakistan	Remote	Re

Total rows: 40 Query complete 00:00:00.950

SQL Queries Used for Verification

```
SELECT * FROM opportunities;
```

```
SELECT COUNT(*) FROM opportunities;
```

```
SELECT category, COUNT(*)  
FROM opportunities  
GROUP BY category;
```

```
SELECT work_mode, COUNT(*)  
FROM opportunities  
GROUP BY work_mode;
```

```
SELECT *  
FROM opportunities  
WHERE status = 'Open';
```

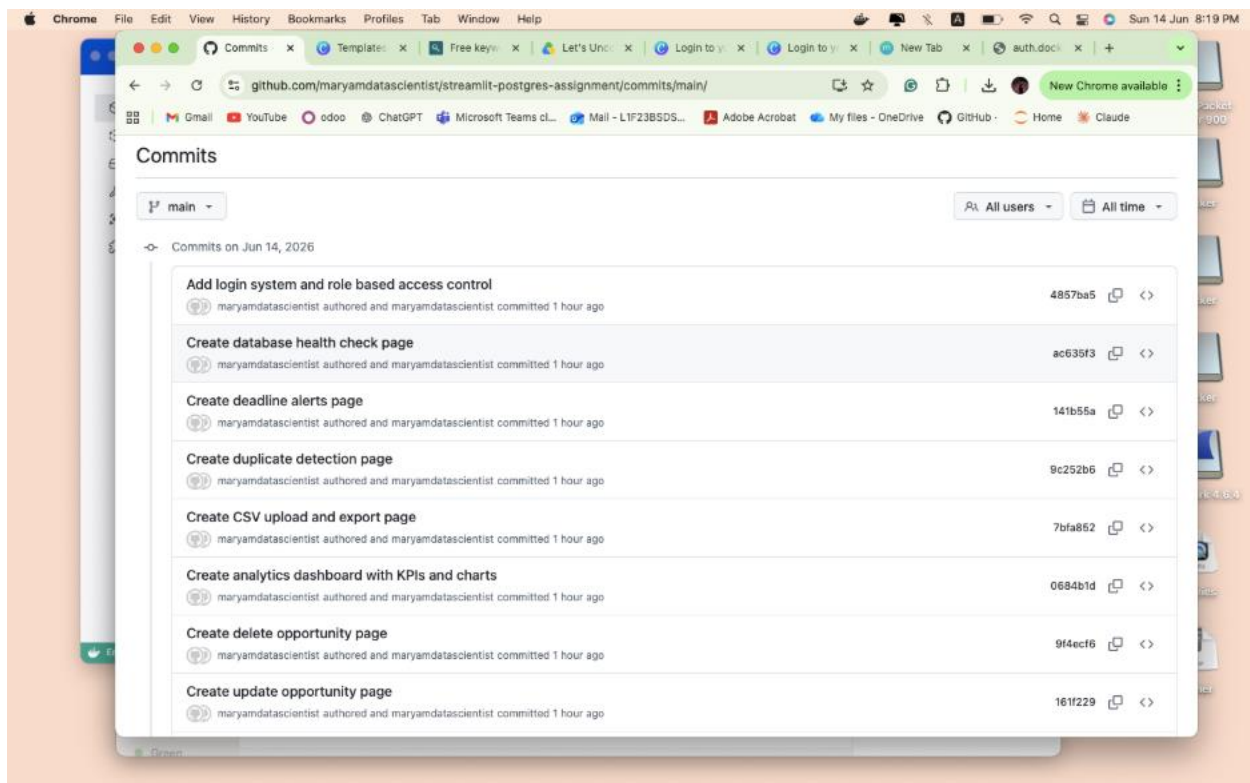
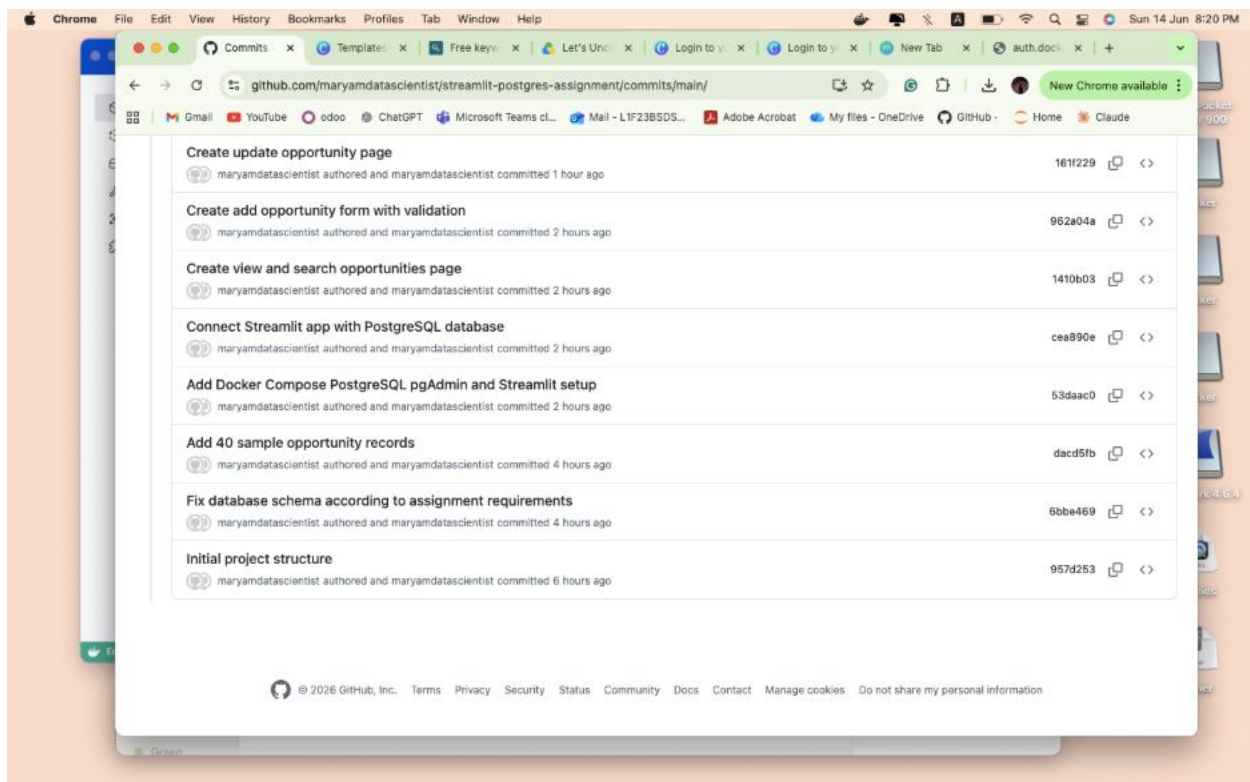
```
SELECT *  
FROM opportunities  
WHERE application_deadline <= CURRENT_DATE + INTERVAL '7 days';
```

These queries confirmed that the table exists, records are inserted correctly, categories and work modes are available, open opportunities can be filtered, and deadline alerts can be verified.

12. GitHub Contribution Evidence

GitHub was used for version control and project collaboration. The repository contains source code, database scripts, Docker files, screenshots, README, and report.

GitHub Commit Screenshot



Example Commits

Commit Message	Purpose
Initial project structure	Created base folder and files
Add 40 sample opportunity records	Added seed data
Add Docker Compose PostgreSQL pgAdmin and Streamlit setup	Added Docker services
Connect Streamlit app with PostgreSQL database	Added database connection
Create view and search opportunities page	Added read/search feature
Create add opportunity form with validation	Added create feature
Create update opportunity page	Added update feature
Create delete opportunity page	Added delete feature
Create analytics dashboard with KPIs and charts	Added dashboard
Create CSV upload and export page	Added CSV functionality
Create duplicate detection page	Added duplicate detection
Create deadline alerts page	Added deadline alerts
Create database health check page	Added DB verification page
Add login system and role based access control	Added Admin and Viewer roles

Contribution Table

Member Name	GitHub Username	Tasks Completed
-------------	-----------------	-----------------

Maryam	maryamdatascientist	setup, Docker Compose, PostgreSQL schema, seed data, pages
Fatima	fatima-5671	CRUD, dashboard, CSV, duplicate detection, deadline alerts,
Syeda Aniq	Aniq-r	seed data, Streamlit pages, login system, documentation

13. Challenges Faced and Solutions

Challenge 1: Docker Desktop Version Compatibility

Docker Desktop initially showed compatibility issues with the macOS version. The solution was to install a compatible Docker Desktop version for the system and verify it using [docker info](#).

Challenge 2: Empty Docker Compose File

The first Docker Compose run showed an [empty compose file](#) message because the [docker-compose.yml](#) file was blank. The file was rewritten with the required PostgreSQL, pgAdmin, and Streamlit services.

Challenge 3: pgAdmin Authentication Error

pgAdmin initially showed a password authentication error for the PostgreSQL user. The issue was solved by using the correct credentials from the Docker Compose file:

```
Username: app_user
Password: app_password
```

Challenge 4: PostgreSQL Host Confusion

When pgAdmin runs inside Docker, the PostgreSQL host must be [postgres_db](#), not [localhost](#). This was corrected in the pgAdmin server registration.

Challenge 5: Streamlit Database Connection

Streamlit needed a clean database connection layer. This was solved by creating `db.py` using SQLAlchemy and environment variables.

Challenge 6: Role-Based Access Control

Admin and Viewer access needed to be separated. This was implemented using Streamlit session state in `auth.py`.

14. Conclusion

The Internship and Job Tracking Dashboard was successfully developed as a complete deployable data application. The system uses Streamlit for the frontend, PostgreSQL for permanent data storage, pgAdmin for database verification, Docker Compose for multi-container deployment, and GitHub for version control.

The final application supports adding, viewing, searching, updating, deleting, uploading, exporting, duplicate detection, deadline alerts, analytics dashboard, database health check, and role-based login.

The project meets the main requirements of the assignment and can be reproduced on another system using Docker Compose and the instructions provided in the README file.